

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 3: Hybrid Systems Verification

Prof. Daniel Holcomb

Submitted by:	Jaideep Khare and Shu-Lin Shih
Student ID:	35193853 and 35199856
Lab No.:	3
Submission Date:	14th April, 2026

Overview

This report documents reachability analysis for hybrid models using **PHAVerLite**, following Lab 3. Hybrid systems combine continuous flows per location with discrete jumps; PHAVerLite uses a piecewise over-approximation of the dynamics so that unreachability results are sound for the concrete model, while reachability of a target in the abstraction does not imply reachability in the original system [Frehse, 2008].

The workflow in each part was: (1) obtain or build a `.pha` model, (2) set partition constraint `pc` and location invariants to bound the search, (3) compute reachable sets and/or run `is_reachable` checks, and (4) relate plots and tool output to the lab questions.

1 Part A: Analysis of Bouncing Ball Model

A.1: Reachable sets for two values of `pc`

Use *PHAVerLite* to compute reachable states with `pc=0.5` and `pc=0.05`. Plot both on the same axes (clearly distinguishable). Answer: for each `pc`, is $(x, v) = (0.55, 0.0)$ unreachable based on the plot?

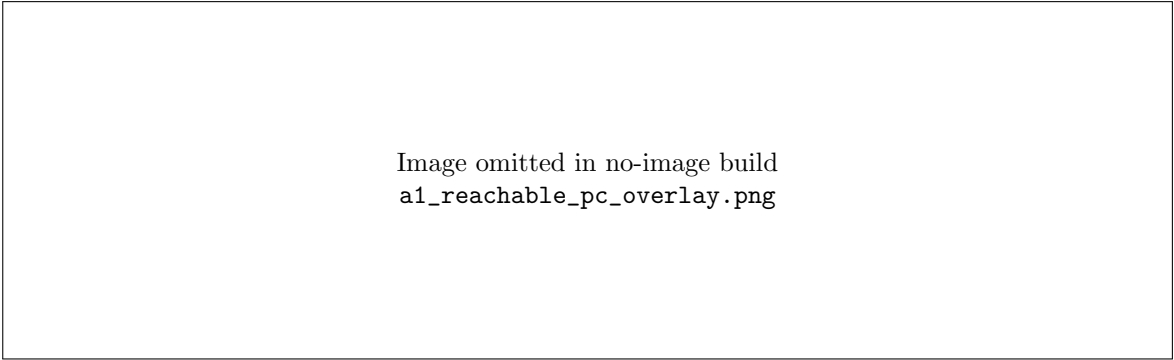


Image omitted in no-image build
`a1_reachable_pc_overlay.png`

Figure 1: A.1: Reachable states for `pc=0.05` overlaid on `pc=0.5` (replace filename after exporting the plot).

Interpretation (plot).

- For `pc=0.5`: (fill in after inspection.)
- For `pc=0.05`: (fill in after inspection.)

A.2: `is_reachable` vs. plot; sweep `pc`

Verify $(0.55, 0.0)$ with `is_reachable` for `pc=0.5` and `0.05` (consistency with A.1). Then sweep `pc` over 0.5, 0.2, 0.1, 0.05, 0.01, and 0.005. If any run exceeds 15 minutes, report time as “timed out.”

Example command form (adjust names to your model):

```
check = bball.is_reachable(target);
```

Partition pc	Condition reachable?	Num locations reached	CPU time (s)
0.5	—	—	—
0.2	—	—	—
0.1	—	—	—
0.05	—	—	—
0.01	—	—	—
0.005	—	—	—

Table 1: A.2 reachability sweep (fill from PHAVerLite output).

Follow-up (no extra runs). If you repeated the reachability check for $pc=1.0$, $pc=0.075$, and $pc=0.001$, which case(s) would *definitely* report the target unreachable? Answer from soundness of unreachability under over-approximation and the table above.

(Write-up goes here.)

A.3: Bounce counter n and peak-height queries

Fix $pc=0.05$. Discuss whether peak height exactly 0.15 on bounces 1–4 appears consistent with the reachable set plot. Add variable n (init, derivatives per location, resets on jumps). Enforce $n \leq 10$ via an invariant. Add four `is_reachable` checks $n1 \dots n4$ for $(x, v) = (0.15, 0)$ on each bounce. Submit `<lastname>_A3.pha`.

```
check = bball.is_reachable(n1);
check = bball.is_reachable(n2);
check = bball.is_reachable(n3);
check = bball.is_reachable(n4);
```

Results vs. plot: (fill in.)

2 Part B: Model of Heating System

B.1: Reachable set with $pc=0.3$

Encode the heating model in PHAVer syntax. Add a clock t (init 0, derivative 1 in both locations, value retained across jumps). Enforce $0 \leq t \leq 10$ in both locations. Plot reachable states: x-axis time, y-axis temperature.

Image omitted in no-image build
b1_heating_reachable_t_x.png

Figure 2: B.1: Reachable states (time vs. temperature) at $pc=0.3$.

B.2: Time-bounded temperature property

From initial (cool, $t=0, x=22$), show temperature goes below 20°C within 0.98 s. Use a counterexample region with $t==0.98$ (not $t>0.98$). Find the largest pc (within 0.1 of the true threshold) for which the bad set is unreachable. Submit `<lastname>_B2.pha`.

Largest pc found: —

Search method: (e.g. bisection / bracketing between reachable and unreachable pc .)

3 Part C: Car Acceleration and Braking

Scenario: two intersections 500 m apart; stop sign then traffic light. Full stop at sign; accelerate at 3.5 m s^{-2} . Braking for red light starts between 250 m and 300 m past the stop sign (i.e. 200 m–250 m from the light). Braking acceleration -5.5 m s^{-2} until stop.

C.1: Hybrid automaton diagram

Circle locations; label guards on jumps; derivatives of position and velocity per location; location invariants (including to force transitions when guards stay true).

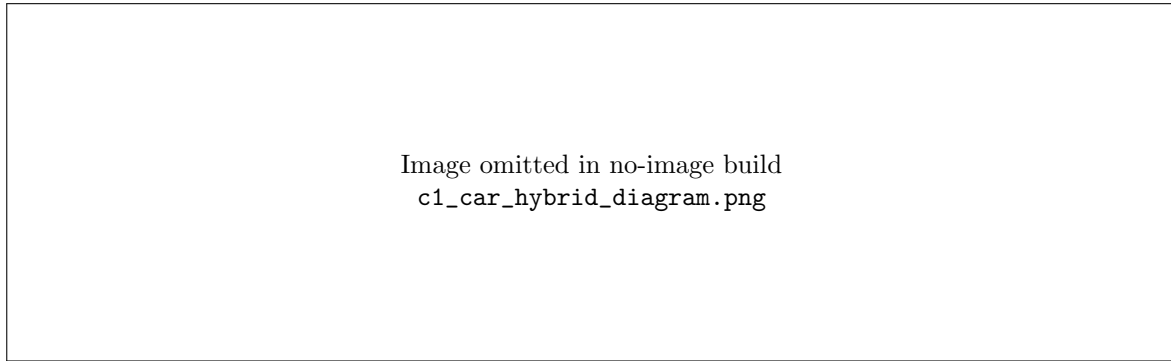


Figure 3: C.1: Hybrid automaton sketch (or export from notes).

C.2: PHAVer model and reachable set

Translate to PHAVer. Use reasonable invariants to bound the state space. With $pc=3.0$, plot reachable states: position (x -axis) vs. velocity (y -axis).

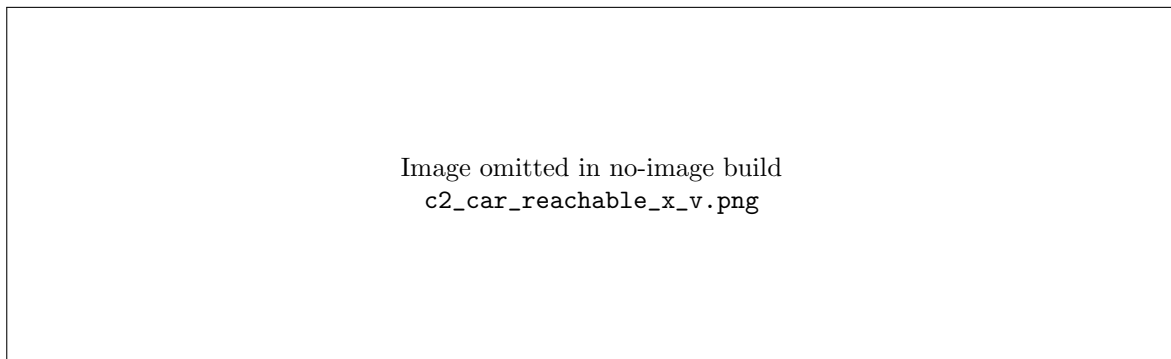


Figure 4: C.2: Reachable states in the (x, v) plane at $pc=3.0$.

C.3: Three reachability properties

For each property below, give the exact `is_reachable(...)` query, include one screenshot with PHAVerLite text output for all three checks, and explain why unreachability (or the appropriate abstraction argument) proves the property true/false. Submit `<lastname>_C3.pha`.

Property	Summary
1	The car will definitely stop before it reaches the 2nd intersection.
2	The velocity cannot be above 46.5 m s^{-1} while braking.
3	The velocity cannot be above 40.0 m s^{-1} while braking.

Table 2: C.3 properties (from lab handout).

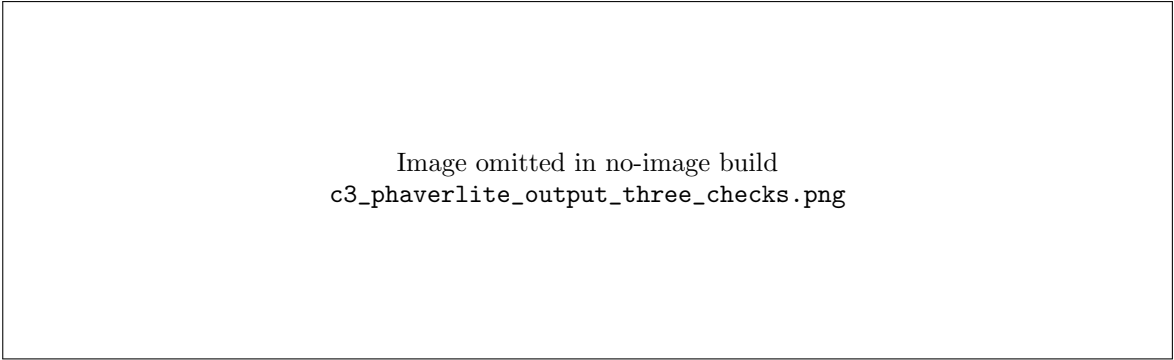


Image omitted in no-image build
c3_phaverlite_output_three_checks.png

Figure 5: C.3: PHAVerLite output for the three `is_reachable` queries.

C.4 (ECE 622 only): 0–60 MPH time estimate

Add a time variable; $60 \text{ mph} = 26.8 \text{ m s}^{-1}$. Use `pc=3.0`. Two queries: (i) 26.8 m s^{-1} unreachable by time T ; (ii) reachable by time $T + 0.1 \text{ s}$. Report an interval for 0–60 time. Submit `<lastname>_C4.pha`.

Estimated 0–60 interval: —

Conclusion

Summarize how coarser vs. finer `pc` affected precision and runtime in Parts A–B, and how over-approximation guided the safety arguments in Part C.

Appendix: Tooling and submission artifacts

This report template was adapted from the Lab 2 `latex-build` layout. Build with `latexmk` (see `README.md`). PHAVerLite models and plots were produced on the course toolchain; figures can be synced via Git alongside this `.tex` file.

Files to submit (replace `<lastname>` per handout):

- `<lastname>_A3.pha`
- `<lastname>_B2.pha`
- `<lastname>_C3.pha`
- `<lastname>_C4.pha` (ECE 622 only)

References

Goran Frehse. PHAVer: Algorithmic verification of hybrid systems past HyTech. *International Journal on Software Tools for Technology Transfer*, 10(3):263–279, 2008. doi: 10.1007/s10009-007-0062-3.